p0874R0
Syntax to anonymously refer to the current declaration contexts

Published Proposal, 2017-11-20

This version:
https://cor3ntin.github.io/CPPProposals/anonymous_context_access/anonymous_context_access.html

Author:
Corentin Jabot

Audience:
EWG, SG7

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

A way to refer to the current class or namespae without naming them.

Table of Contents

§ 1. Introduction

There are cases where referring to a class/struct from within itself is either cumbersome (macro definition), or not possible (anonymous entities). It also forces the code injection/metaclass proposal to inject a name in a class or namespace code fragment, in order to refer to the injected declaration context.

This proposal introduces a syntax to refer to the class, struct, union or namespace in which the syntax is used, independant of the entity’s name.

§ 1.1. Anonymous struct / class

There has been a strong interest for being able to provide constructors and destructors for anonymous classes and structs.

§ 1.2. Resolving ambiguity

Having a way to refer to both the current class and the current namespace offers a simple way to resolve ambiguities between a name that exist both in a class and its enclosing namespace.

```
namespace long::namespace_declaration::foo {
    constexpr static int bar = 0;
    class foo {
        constexpr static int bar = 1;
        void f() {
            static_assert(typename(namespace)::bar == 0);
        }
    };
}
```

§ 1.3. Code Injection

[p0712r0] proposes to introduce a name in a code fragment to refer to the entity in which it will be injected. For example `-> namespace Foo { };` use the placeholder `Foo` to refer to the the class in which it will be injected. This has a few drawbacks. The developer needs to come with a placeholder name, which will probably end up being `C` or `N` most of the time. It make the intent of the code less clear to a reader. Is `Foo` the name of the code-fragment ? are we created an object `Foo` in the current scope ?

These issues can be solved if we have a consistent way to refer to an anonymous class or a current namespace.

§ 1.4. Macros

Macros that define or declare class constructors, destructors or otherwise manipulate a class' name from inside the class are quite frequent. Such macros are often used to avoid duplicating repetitive code stubs, and have been found in large projects such as LLVM and Chromium.

As they rely on knowing the class name, they have to been defined as `#define EvilMacro(ClassName /*,...*/)` and used in the same fashion.

Having a consistent way to refer to a class name without an identifier would simplify the use of such macros.

And while defining class members in macros is certainly not something that should be encouraged, it has its uses in large code base.

Code injection will certainly make the need for such macros less frequent. However, it has been identified that a facility akin to the one this paper proposes is one of the features required for The Qt

framework to benefit from code injection without having to modify millions of lines of code. That is one of the driving motivation behind this proposal.

§ 2. Syntax

`typename(namespace)` is valid in a scope identifier and refers to the current namespace, or the global namespace.

As a scope identifier, `typename(class)` refers to the identifier of the current `class`, `struct`, or `union`.

`typename(class)` can refer to the declaration of a constructor, and `~typename(class)` can similarly refer to the declaration of the class destructor.

Constructors or destructors declared using this syntax or the class of the name belong to the same overload set (and therefore can not be defined with one syntax and later redefined with the other syntax).

When it's not used as the destructor or constructor name, `typename(class)` refers to the type of the `class`, `struct`, or `union` in which it is used.

In the rest of this document, a ***typename id expression*** refers to either `typename(class)` or `typename(namespace)`.

Using `typename(class)` outside of a `class`, `struct` or `union` is ill-formed.

Constructors and destructors using this syntax in unions, whether they are anonymous or not, remains invalid.

As a typename id expression unambiguously refers to the entity in which it is used, whether that is the current namespace, or the current record, it can not be preceded by a nested-name-specifier, nor can it be used through an object. The following expressions are therefore ill-formed:

`::typename(namespace)` `Foo::typename(class)` - ill-formed because a scope-specifier is specified.

`foo->typename(class)/*...*/` : ill-formed, call through an object.

§ 2.1. Templates

In a template class definition, `typename(class)` behaves like the *injected-class-name*. `typename(class)` can be followed by a template arguments list, like a type-id.

```
typename(class)<Foo> /*...*/
```

§ 3. Code injection

While this syntax is not directly related to code injection, code injection may benefit from it.

In a code fragment `typename(class)` and `typename(namespace)` should refer to the scope the fragment will be injected in, alleviating the need to inject names in the scope of the code fragment for that purpose.

[p0712r0] suggests using `typename(reflection-expression)` to get a type from a reflection. While I believe there are arguably better approaches (including using the same syntax for code injection and "reversing" a reflection), this proposal would not prevent that syntaxe, and it would be cohesive and logical

```
typename(class)
typename(namespace)
typename(reflection-expression)
```

These three expression return, depending on the context, either a type, or a scope specifier.

[p0712r0] further suggests `namespace(reflection-expression)` as a mean to get a scope specifier from a reflection on a namespace. The same semantic could be achieve with the typename(reflection-expression) syntax, depending on the context.

§ 4. Alternatives approaches

Some GCC versions allow `decltype(*this)` to be evaluated in the context of a static member declaration. This can be used to the same effect than the proposed `typename(class)`. However this non standard behaviour relies on an unnatural semantic and does not solve the problem of addressing the current namespace. It also requires introducing a static member in the class for the purpose of querying the class type.

Different names have been discussed, including `decltype(class)`, `typename(typename)` `namespace(namespace)` or the introduction of new keywords like `__classname`. However these syntaxes either make no sense semantically (`decltype(class)`), are absurdly convoluted or are __ugly.

`(class)` and `(namespace)` could also be viable, more terse expressions. However that requires expressions such as `decltype((class))` to have 2 pairs of parentheses which may be confusing and error prone.

5. Implementation

A proof-of-concept (for the `typename(class|namespace)` syntax) was implemented in clang and is available on Github. No particular difficulty was encountered.

Index

Terms defined by this specification

References

Informative References

[P0712R0]
Andrew Sutton, Herb Sutter. Implementing language support for compile-time programming. URL: https://wg21.link/p0712r0